Task 1: Guided Activity, Library lending model

```
import Foundation

protocol Borrowable {
    var id: Int { get }
    var title: String { get }
    var isAvailable: Bool { get set }
}

struct Book: Borrowable {
    let id: Int
    let title: String
    let author: String
    var isAvailable: Bool = true
}

struct Magazine: Borrowable {
    let id: Int
    let title: String
    let issue: Int
    var isAvailable: Bool = true
}

enum LibraryError: Error {
    case itemNotFound(id: Int)
    case alreadyBorrowed(title: String)
    case limitReached(max: Int)
}

final class Member {
    let name: String
    private(set) var borrowedIDs: [Int] = []

    static let limit = 2

    init(name: String) {
```

```
        self.name = name
    }

    func recordBorrow(_ id: Int) throws {
        guard borrowedIDs.count < Member.limit else {
            throw LibraryError.limitReached(max: Member.limit)
        }

        borrowedIDs.append(id)
    }

    func recordReturn(_ id: Int) {
        borrowedIDs.removeAll { $0 == id }
    }
}
struct Catalog<Item: Borrowable> {
    private var items: [Int: Item] = [:]

    mutating func add(_ item: Item) {
        items[item.id] = item
    }

    mutating func checkout(id: Int, to member: Member) throws {
        guard var item = items[id] else {
            throw LibraryError.itemNotFound(id: id)
        }

        guard item.isAvailable else {
            throw LibraryError.alreadyBorrowed(title: item.title)
        }
```

```
        guard member.borrowedIDs.count < Member.limit else {
            throw LibraryError.limitReached(max: Member.limit)
        }

        try member.recordBorrow(id)

        item.isAvailable = false
        items[id] = item

        print("\(item.title) borrowed successfully by \(member.name).")
    }
}

var catalog = Catalog<Book>()

catalog.add(Book(id: 1, title: "Swift Basics", author: "John"))
catalog.add(Book(id: 2, title: "iOS Development", author: "David"))
catalog.add(Book(id: 3, title: "Swift Programming", author: "Sam"))

let member = Member(name: "Asha")

do {
    try catalog.checkout(id: 1, to: member)
} catch {
    print("Error:", error)
}
do {
    try catalog.checkout(id: 1, to: member)
} catch {
    print("Error: The book is already borrowed.")
}
```

```
do {
   try catalog.checkout(id: 2, to: member)
} catch {
   print("Error:", error)
}
do {
   try catalog.checkout(id: 3, to: member)
} catch LibraryError.limitReached(let max) {
   print("Error: Borrowing limit reached. Maximum: \(max)")
} catch {
   print("Error:", error)
}

print("Borrowed IDs:", member.borrowedIDs)
```

output:

Output

```
ERROR!
Swift Basics borrowed successfully by Asha.
Error: The book is already borrowed.
iOS Development borrowed successfully by Asha.
Error: Borrowing limit reached. Maximum: 2
Borrowed IDs: [1, 2]
```

Task 2: Practice Task, Generic repository:

```
import Foundation
protocol Entity {
   var id: Int { get }
}
enum RepositoryError: Error {
   case duplicateID(Int)
   case notFound(Int)
```

```
}
struct Repository<T: Entity> {
    private var storage: [Int: T] = [:]

    mutating func add(_ item: T) throws {
        guard storage[item.id] == nil else {
            throw RepositoryError.duplicateID(item.id)
        }

        storage[item.id] = item
    }

    func find(_ id: Int) -> T? {
        return storage[id]
    }

    mutating func remove(_ id: Int) throws {
        guard storage.removeValue(forKey: id) != nil else {
            throw RepositoryError.notFound(id)
        }
    }

    func all(where predicate: (T) -> Bool) -> [T] {
        return storage.values
            .filter(predicate)
            .sorted { $0.id < $1.id }
    }
}
struct Student: Entity {
    let id: Int
    let name: String
```

```
    let marks: Int
}
struct Course: Entity {
    let id: Int
    let title: String
}
var students = Repository<Student>()

do {
    try students.add(Student(id: 3, name: "Meena", marks: 90))
    try students.add(Student(id: 1, name: "Asha", marks: 85))
    try students.add(Student(id: 2, name: "Ravi", marks: 70))
    try students.add(Student(id: 1, name: "Divya", marks: 95))
} catch RepositoryError.duplicateID(let id) {
    print("Error: Student ID \(id) already exists.")
} catch {
    print("Error:", error)
}
if let student = students.find(1) {
    print("Found:", student.name, student.marks)
}
let toppers = students.all { $0.marks >= 80 }

print("Students scoring 80 or more:")

for student in toppers {
    print("\(student.id): \(student.name) - \(student.marks)")
}
do {
    try students.remove(2)
    print("Student removed successfully.")
```

```
} catch {
    print("Error:", error)
}

var courses = Repository<Course>()

do {
    try courses.add(Course(id: 101, title: "Swift"))
    try courses.add(Course(id: 102, title: "iOS Development"))
} catch {
    print("Error:", error)
}

print("Courses:")

for course in courses.all(where: { _ in true }) {
    print("\(course.id): \(course.title)")
}
```

Output

```
ERROR!
Error: Student ID 1 already exists.
Found: Asha 85
Students scoring 80 or more:
1: Asha - 85
3: Meena - 90
Student removed successfully.
Courses:
101: Swift
102: iOS Development
```

Task 3: Extension Task, Reports through protocol extensions

```
import Foundation

protocol Entity {
var id: Int { get }
}

protocol Describable {
var summary: String { get }
}

enum RepositoryError: Error {
case duplicateID(Int)
case notFound(Int)
}

struct Repository<T: Entity> {
private var storage: [Int: T] = [:]

```
mutating func add(_ item: T) throws {
   guard storage[item.id] == nil else {
      throw RepositoryError.duplicateID(item.id)
   }
   storage[item.id] = item
}

func find(_ id: Int) -> T? {
   return storage[id]
}
```
```

```
mutating func remove(_ id: Int) throws {
    guard storage.removeValue(forKey: id) != nil else {
        throw RepositoryError.notFound(id)
    }
}

func all(where predicate: (T) -> Bool) -> [T] {
    return storage.values
        .filter(predicate)
        .sorted { $0.id < $1.id }
}
```

```
}

struct Student: Entity, Describable {
let id: Int
let name: String
let marks: Int

```

```
var summary: String {
    return "Student ID: \(id), Name: \(name), Marks: \(marks)"
}
```

```
}

struct Course: Entity, Describable {
let id: Int
let title: String
```

```
var summary: String {
   return "Course ID: \(id), Title: \(title)"
}
```

}

extension Repository where T: Describable {
func printReport() {
print("===== Repository Report =====")

```
   for item in all(where: { _ in true }) {
      print(item.summary)
   }

   print("==============================")
}
```

}

var studentRepo = Repository<Student>()

do {
try studentRepo.add(Student(id: 1, name: "Asha", marks: 90))
try studentRepo.add(Student(id: 2, name: "Ravi", marks: 75))
} catch {
print("Error:", error)

```
}

studentRepo.printReport()

var courseRepo = Repository<Course>()

do {
try courseRepo.add(Course(id: 101, title: "Swift"))
try courseRepo.add(Course(id: 102, title: "UIKit"))
} catch {
print("Error:", error)
}

courseRepo.printReport()
```

output:

Output

```
===== Repository Report =====
Student ID: 1, Name: Asha, Marks: 90
Student ID: 2, Name: Ravi, Marks: 75
============================
===== Repository Report =====
Course ID: 101, Title: Swift
Course ID: 102, Title: UIKit
============================
```